

Weekly Project Log

Week 1 — 2026-06-13 to 2026-06-19

13 June. I set the general direction of my EPQ. The repository's report, research notes, data, code, appendix, presentation and Production Log folders date from this stage. My first idea - using machine learning to predict cryptocurrency - was too broad because it did not name the asset, the target or the way I would judge the models.

17 June. I narrowed the project to Bitcoin volatility forecasting. I chose volatility instead of trying to predict the exact Bitcoin price because volatility has a clearer connection to risk and can be compared using numerical errors. The research notes from this stage cover log returns, realised volatility, rolling historical volatility and GARCH. For the machine-learning side, I chose Random Forest and LSTM because they use different ideas: Random Forest can learn nonlinear relationships from prepared features, while LSTM is designed for sequences.

I originally planned to use Yahoo Finance, but I changed the data source to Hyperliquid. This gave the project daily OHLCV data from one specific Bitcoin perpetual-futures market. The saved files include the first dataset, derived daily log returns, a rolling-window volatility target and an initial results table.

I chose not to rank the models by error alone. I set four comparison dimensions: error, explainability, runtime and whether the extra complexity gave a useful improvement.

Why this mattered. Defining the asset, target and comparison methods made the project testable. The target definition controls what every model is asked to predict.

Next recorded task. Prepare the data more carefully, build the baseline methods first and then compare them with the machine-learning models using the same dates and the same error measures.

Week 4 — 2026-07-04 to 2026-07-10

9-10 July. The repository records a Hyperliquid daily-data refresh and a new model run so that the report did not rely on the earlier dataset. Saved checks cover date order and price fields; the derived outputs contain recalculated log returns and the 30-day volatility target.

The main comparison covered rolling historical volatility, GARCH, lagged linear regression and Random Forest. It used a time-based split rather than randomly mixing the rows because a forecasting model should only learn from information available before the date being predicted. The saved evaluation compares the models using MAE, MSE and RMSE, and the corresponding results section and comparison chart record that run.

The error table alone did not explain why the methods differed, so the report includes a short explanation of each one. Rolling volatility was the simplest baseline. GARCH modelled volatility clustering. Linear regression checked whether the lagged features already contained useful information, while Random Forest tested nonlinear relationships.

Why this mattered. A data refresh can change the figures, so the report and saved outputs must refer to the same run. A shared test period kept the model results comparable.

Next recorded task. Complete the LSTM model, check the date alignment of every prediction and add more tests to see whether the model ranking was stable.

Week 5 — 2026-07-11 to 2026-07-17

12 July. A further saved refresh included a daily candle that was still open, so its closing price, volume and trade count were incomplete. The data step was changed to retain a candle only after its end time had passed.

13 July. The repository records a completed LSTM comparison and separates the code into data preparation, features, models, evaluation and outputs. This structure supports repeating the same process without manually changing several files.

The GARCH predictions were being matched by reset row number rather than by date. A forecast could therefore be compared with the wrong target day. The output was changed to match by date; the saved model outputs and affected report sections record the corrected rerun.

14 July. Further saved checks identified the order used in the GARCH update, which was corrected so that the current day's movement did not affect its own forecast. The rolling-volatility calculation was also checked, and the recorded LSTM scaling is fitted only on the earlier training data, not on the later validation period.

After these corrections, the saved robustness outputs compare 14-day and 30-day volatility targets, earlier and later test halves, low-, medium- and high-volatility periods and four expanding time windows. They also include three LSTM seeds and Random Forest feature importance. Their purpose is to test whether the main conclusion depends on one particular setting.

Why this mattered. A result can look reasonable even when its dates or calculations are wrong. The recorded corrections changed the ranking, so the final comparison had to use the corrected run.

Next recorded task. Run the complete pipeline one more time with the latest finished daily data and use the checked results for the final discussion and conclusion.

Week 6 — Beginning 2026-07-18

20 July. The final saved data refresh returned 1,241 Hyperliquid daily rows. Excluding the one day still in progress left 1,240 completed daily candles ending on 2026-07-19. The saved data-quality results cover missing fields, date order, daily spacing, positive prices, OHLC consistency, volume and trade count.

After feature and volatility-target preparation, the final saved modelling table contained 1,195 rows. The recorded split uses 950 earlier rows for training and 245 later rows for testing. The test period started on 2025-11-16, so the models were compared on the same later section of the data without random shuffling.

The final 30-day RMSE ranking was:

1. GARCH(1,1): 0.00098502
2. Lagged linear regression: 0.00140087
3. Rolling historical volatility: 0.00142744
4. LSTM: 0.00174351
5. Random Forest: 0.00232370

GARCH also ranked first with the 14-day target and in the repeated expanding-window tests. The LSTM and Random Forest did not beat the simpler rolling baseline overall. For this dataset and setup, their extra complexity was not justified by better accuracy.

The saved automated test run reports that all 39 tests passed. The discussion and conclusion use the checked results and limit the claim to the tested Bitcoin market, dates and machine-learning pipelines.

Why this mattered. The final result depended on correct dates, completed market data and keeping later information out of earlier stages of the model.

Next recorded task. Finish the presentation, explain the main corrections and results clearly, and prepare for questions about why the machine-learning models did not perform better.

Week 7 — 2026-07-25 to 2026-07-26

25-26 July. I tightened the research question so that it matched the recorded prediction task: the next day's update to a volatility measure based on Hyperliquid BTC perpetual-futures returns. Consecutive 30-day targets share 29 returns, so this is not a forecast of a completely new 30-day period. I also set the aim, six objectives and four points for judging the models: accuracy, robustness, interpretability and practicality.

The model comparison also changed. The project-local Random Forest was replaced with scikit-learn's implementation. Six Random Forest settings and four LSTM settings were compared using only the end of the training period, without using the final test set to choose them. RMSE and MAE were kept, and QLIKE was added using squared target and forecast values with epsilon $1e-12$.

After the full rerun, the ranking stayed the same. GARCH's RMSE and QLIKE were 0.00098502 and 0.00370047. LSTM's RMSE was 0.00174351. Random Forest improved to 0.00220594, but it still did not beat rolling. The outputs also gained linear coefficients, an LSTM input-removal check and a comparison of dependencies, tuning work and runtime. All 43 automated tests passed.

Why this mattered. Using the same dates does not make the models identical because their inputs and target conversions also differ. The comparison covers the tested pipelines, not an isolated effect of model type.

Next recorded task. Rehearse the presentation, check the PDF layout and transfer only statements supported by the candidate's own decisions or personal verification into the centre-issued Production Log form.

Week 8 — 2026-08-06

Repository update, 6 August. The appendix was checked for six items: a detailed timetable, Gantt chart, source evaluation, report-structure mind map, risk assessment and data charts. Four were already present. The Gantt chart and mind map were missing, and the timetable did not clearly separate planned dates from completed work.

The timetable was expanded, the two missing diagrams were added and the six items were put into one appendix pack. The Production Log was also checked for both what changed and why. One dated record was added for the new appendix work instead of rewriting the earlier history.

The first Word export flattened some tables and pushed wide ones outside the page. The export was changed to keep real tables inside the A4 text area, and the English and Chinese files were checked page by page. The project still passed 43 automated tests and 192 bundle checks. AI assistance was used for code review and editing, data processing and checks, automated tests, writing and editing, translation, and Word/PDF export. Repository files and timestamps show saved work, not who performed each action. The final declaration must distinguish AI-assisted work from the candidate's own decisions and any verification the candidate personally carried out.

Why this mattered. The appendix should let the examiner find the evidence quickly. A Gantt chart records the plan, but it does not prove that every task happened on the planned date.

Next recorded task. Check dates and every first-person statement against personal evidence, complete the centre-issued form and leave the supervisor and presentation sections blank until the real entries exist.